Crossbot: bit-parallel constraint search and a remaining dense-grid bottleneck

A reproducible engineering study of the shipped implementation

Code baseline: `d9e568f` · Measurements: `benchmarks/results.json`

Abstract. Crossbot validates and fills fixed crossword grids with distinct dictionary words and optional thematic locks, then independently verifies every returned assignment. On an Apple-vendor aarch64 Linux VM using CPython 3.14.7, the default bitset maintaining-arc-consistency solver filled 87 of 88 supplied grids within a five-second budget; all 87 successful seed-zero runs took less than one second. Core grid 87 remains unresolved, so the prescribed all-grid exit condition is not met. We traced the solver's mechanisms to established constraint-search, bitset, restoration, randomization and propagation literature, explicitly recording where the exact engineering realization has no located origin. An isolated study on three known-fill grids with three seeds each is consistent with arc consistency, value ordering and restarts contributing more than support-signature caching or degree ties. Forward checking has lower overhead on easy instances but more timeouts. We claim neither a new algorithmic principle nor state-of-the-art performance: no controlled external-system comparison was performed. Stronger reasoning and structure-aware search on the unresolved dense instance remain open. The report preserves failures, negative exploratory results and verified witnesses rather than substituting a hidden timeout or precomputed fill.

1 Context

1.1 Problem and scope

A fixed grid defines a variable for each maximal horizontal or vertical run of at least two open cells. Its domain is the set of dictionary words of the required length. A crossing equates two indexed characters. Distinctness applies globally, including locked words. A lock fixes one variable to an explicitly normalized word, even if that word is absent from the dictionary. The grid is filled only if every open cell is covered by assigned slots; an isolated open cell therefore makes a fill impossible, rather than inviting an arbitrary extra letter. These are the contracts in `crossbot.spec.md` and the checks in `grid.py`, `problem.py` and `verify.py`.

The report concerns **filling**, not discovering a grid pattern, writing clues, or judging word quality. Membership in these supplied dictionaries is the only lexical-quality criterion. Alphabet handling preserves ñ as a distinct letter. YAML metadata, canonical numbering, lock consistency, coverage and the final assignment are all validated. The suite exercises ordinary, unlocked filling; the thematic and Spanish-letter paths are covered by tests, not by the main English performance campaign.

The goal requests a solver capable of filling every supplied grid with at least one supplied dictionary in less than five seconds, and at least ninety percent below one second. The measured result reaches the second aim in the solver-time metric but not the first condition. This is an **interim analysis of a shipped implementation**, not a certificate that the goal has been reached. Loading and indexing are accounted for separately in Section 4; cold-process compiler performance has not been measured suite-wide.

1.2 Complexity

The general crossword construction/filling decision problem is a classical NP-complete problem in Garey and Johnson's catalogue [1]. An explicit assignment is a polynomial-size certificate and is checked by testing lengths, letters, membership and distinctness. Gourvès and colleagues revisit the grid-graph formulation and prove hardness even under severe structural restrictions when word reuse is prohibited [2]. Their reuse-allowed treewidth algorithms do not automatically apply to crossbot's global distinctness rule. These general results neither prove that this finite suite is difficult nor prove the unresolved instance unsatisfiable. The supplied fixed alphabet and fixed collection are an experimental domain, not an asymptotic hardness argument.

1.3 Prior construction and filling systems

Our earliest recovered primary construction publication is Mazlack's probability-based conference paper [3]; his subsequent system organizes precedence relationships to guide construction [4]. We do not assert that these are historically the first programs: the search recovered them, not a complete chronology of unpublished or earlier work. XENO supports computer-assisted compilation and interactive construction [5]. Ginsberg, Frank, Halpin and Torrance study search choices specifically through crossword instances [6]; this is a closer predecessor to fixed-pattern filling than unconstrained puzzle generation.

PROVERB generates clue-dependent candidate answers and combines them probabilistically over intersections [7]. WebCrow adds web-derived candidate retrieval to the same shared-letter core [8]. Dr.Fill represents clued puzzles as singly weighted constraint problems, with discrepancy search, postprocessing and partitioning [9]. The Berkeley Crossword Solver combines neural candidate generation, loopy belief propagation and local search [10]. French WebCrow demonstrates multilingual candidate experts [11]. Saha and colleagues combine language-model clue answers with full-grid search [12]. The more recent reactive construction study of Biemer and Cooper addresses difficulty during puzzle generation rather than the exact fixed-grid contract used here [13]. These are characterized as related systems, **not** as controlled performance baselines; their objectives, candidate domains, corpora and hardware differ.

1.4 Questions answered by this report

- What is actually implemented, and how can the default strategy be reproduced? Section 2.
- Which ideas are established and which exact realizations remain untraced? Section 3.
- What do complete-suite, cross-solver and isolated-feature measurements establish? Section 4.
- Is any novelty or state-of-the-art claim justified? Section 5.
- Which changes are supported now, and which need a new experiment? Section 6.
- What has been achieved, and what prevents declaring the goal complete? Section 7.

2 The System

2.1 Geometry and dictionary representation

`Grid` records the pattern, canonical row-major keys, ordered slots and bidirectional indexed crossings. A key with two directions emits horizontal before vertical, so a slot index is not the same as a key number. Grid schemas reject unknown fields, malformed shapes, impossible key positions and metadata mismatches. Derived density has a tolerance of 0.01 percentage points; the stated informational word count is not validated. The canonical direction **sets** are compared in key order, and duplicate directions are rejected.

Dictionary lines are stripped and lowercased; empty lines and any line outside `a–z, ñ` are discarded. Unlike locks, dictionary lines are not accent-stripped. The surviving input order, including duplicates, is retained for the reported load count. A separate stable first-occurrence sequence assigns distinct word IDs by length. All IDs of a given length use the same index, including dictionary-exempt locked words appended for this problem. Consequently a word bit means the same thing in all same-length slots, a necessary invariant for distinctness pruning.

For each length, position and alphabet letter, `WordIndex` stores an integer mask whose set bits identify words with that letter at that position. Compilation first writes packed bytearrays and converts them to integers once; it does not repeatedly grow a large integer per word. An index is built lazily and reused by later problems with the same dictionary. With non-dictionary locks only the affected lengths are rebuilt. Dictionary membership also has an independent frozen set for verification.

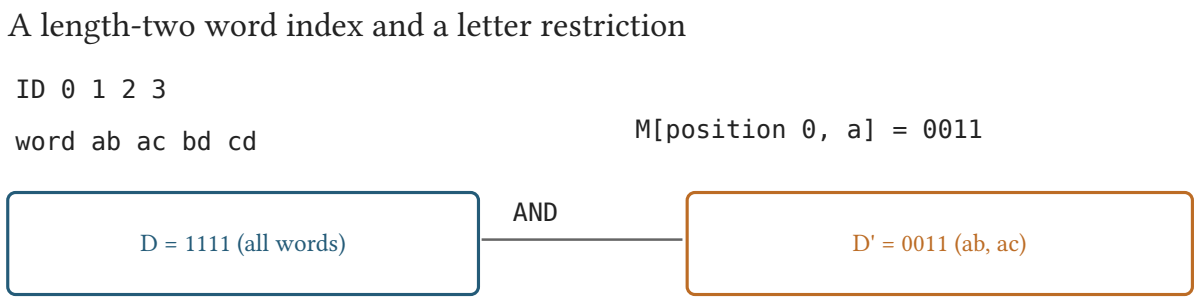


Figure 1: F1. The central integer-domain representation and its elementary intersection, using the concrete toy index `ab, ac, bd, cd`. This is an analytic example, not a timed run; dictionary, seed, budget and machine are inapplicable. Bit strings are exact; no zero or censoring convention is needed.

`Problem` contains geometry, the original lexicon, locks, per-slot indexes, initial domains and same-length peers. A lock is a singleton domain. If a required length has no dictionary word and no lock-provided word, construction raises an input error. Locks are case-insensitive, normalized to canonical Unicode before accent removal, and preserve ñ; their original specs are quoted in errors. Dictionary exemption applies to the locked assignment, not to an arbitrary fabricated word elsewhere.

2.2 Shared propagation and restoration

`Domains.values[i]` is the live word bitset. A trail entry saves `(slot, old_domain, old_support_signatures)` before every change. A mark is a trail length; restoration pops to that length. Support signatures are 27-bit sets, one per slot position, initially unknown (`-1`). Root propagation and every assigned branch run a FIFO queue with a membership set that prevents duplicate queue entries.

For a queued source, a singleton removes its word bit from every same-length peer. A zero domain fails immediately. For each crossing from source position p to target position q, compute the set S of letters with a nonempty intersection between the source domain and `mask[p, letter]`. A singleton uses its actual character instead of scanning the alphabet. If S equals the saved signature, that crossing needs no new filtering: the target has already been restricted to those letters. Otherwise save S, form the union of target masks for supported letters, and intersect the target domain. When fewer letters are absent than present, form the union of **absent** masks and remove it instead. The target is requeued only if its domain changes. This is arc consistency specialized to shared-letter equality, not an implementation of general table GAC.

The support cache is branch state. It is restored with domains; forgetting to restore it can incorrectly skip a needed revision after backtracking. Same-length peers share word IDs; assigning IDs from inconsistent indexes could incorrectly remove a different word. The tests explicitly check both representation invariants. Distinctness filtering only removes singleton values; it does **not** enforce Hall sets or Régin's matching-based all-different consistency [14].

2.3 The default search, with all shipped choices

The default registered solver is `mac` (maintaining arc consistency). `forward` uses the same engine but skips crossing propagation from nonsingleton sources. The former is selected as default because the measured hard cases favour its reliability, not because it wins every timing comparison. There is no worker pool, structural partition, staged compilation solver, precomputed solution, persistent fill cache, or grid-ID special case.

At each depth, check the absolute `perf_counter()` deadline. Select a nonsingleton slot with minimum domain cardinality; ties prefer the greatest number of currently nonsingleton crossing neighbours, then the lowest slot index. Assigned singletons are excluded. If all slots are singleton, return their words. Before search, coverage is checked so an otherwise unconstrained isolated open cell cannot produce an invalid "solution". An all-blocked grid has a valid empty assignment.

Values are ordered as follows; these details define the implementation, including random draws.

1. The sample limit is **48**. If the domain cardinality is at most **96**, enumerate all live IDs and shuffle them.
2. Otherwise, if at least one eighth of IDs up to the highest live bit are present, repeatedly draw a uniform integer in that range and retain distinct live IDs until 48 have been collected. Sort their set before shuffling, so hash iteration does not affect reproducibility.
3. For sparser domains, enumerate live IDs once and use `random.sample` to draw 48 without replacement.
4. Score each sampled word by $\sum_e \ln n_e$, where n_e is the count of target-domain words supporting that word's letter on crossing e. A zero count gives negative infinity. Sort descending, stably preserving randomized order on equal scores. The score approximates the product of independent neighbour supports; it is not belief propagation or a count of globally extendable assignments.
5. Try those sampled values first, then every unsampled live ID in ascending order. Sampling prioritizes values; it does not delete unsampled candidates.

A value assignment saves a mark, makes the selected domain singleton, propagates, then recurses. A failure restores to the mark and increments `backtracks`. The `nodes` statistic counts attempted word assignments, not recursive calls or grid cells. `revisions` counts crossing-domain reductions, excluding distinctness deletions. These definitions matter when comparing MAC with forward checking.

The first search attempt has an allowance of **400 nodes**. A quota exhaustion raises a restart rather than unsatisfiability. The original propagated root domains and support signatures are restored, while the RNG stream continues. After restart number r, the next allowance is $400 \times 2^{\min(r,12)}$ additional nodes. Statistics accumulate over attempts; quotas saturate at 1,638,400 nodes. Quota saturation is not exercised by the recorded suite or ablations. Individual uninterrupted DFS attempts enumerate all values; the restarting algorithm with a finite deadline is not a complete decision procedure. Only exhaustion of an entire attempt can return `unsatisfiable`.

```
propagate root; preserve root domains and signatures
repeat:
    DFS:
        check deadline; choose nonsingleton MRV slot
        if none: return all singleton words
        for value in sampled-LCV order, then remaining IDs:
            check deadline and cumulative node quota
            save trail mark; assign; propagate
            if consistent and recursive DFS succeeds: return fill
            restore mark; count backtrack
        return exhaustive failure
    on quota: restore root; increase next node allowance
    on deadline: return timeout
```

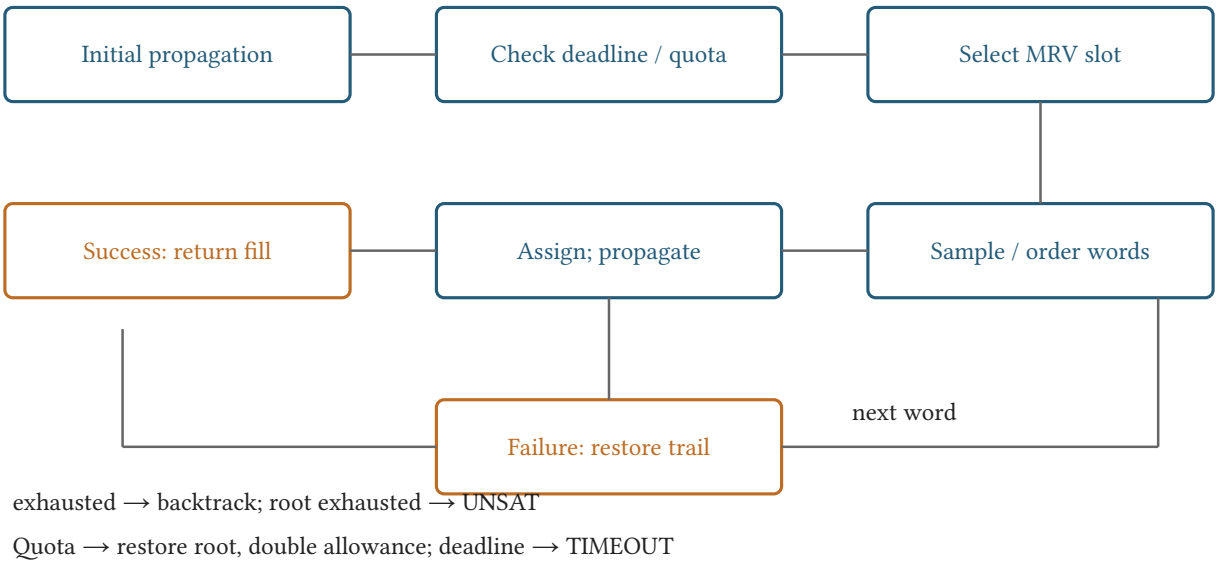


Figure 2: F2. Control-flow summary of the default strategy. A consistent but incomplete branch returns to selection; success is emitted only when every domain is singleton. Quota exhaustion is not an UNSAT proof. This is a code-derived schematic, with no measured dictionary/seed/budget/machine or zero/censored values.

2.4 Frontends and verification boundaries

A registry holds a name, description, callback and default marker for each strategy. It rejects duplicate names or multiple defaults. The CLI and benchmark depend only on that contract. They report mandatory solver, seed and elapsed time fields plus the stable solver-specific keys `nodes`, `backtracks`, `revisions`, `restarts`. Time is solver wall time, not process startup, dictionary loading, problem preparation or verification.

A returned fill is checked against the original problem, not against reduced bitsets: slot coverage and lengths, original dictionary membership, exact lock equality, crossings and no repeats. Invalid fills never reach grid rendering or count as benchmark successes. A lock-violation error is distinguished from other fill defects. Both frontends print ordinary Unicode tables without a terminal-dependent layout. The benchmark builds each problem once and verifies every solver/seed result independently.

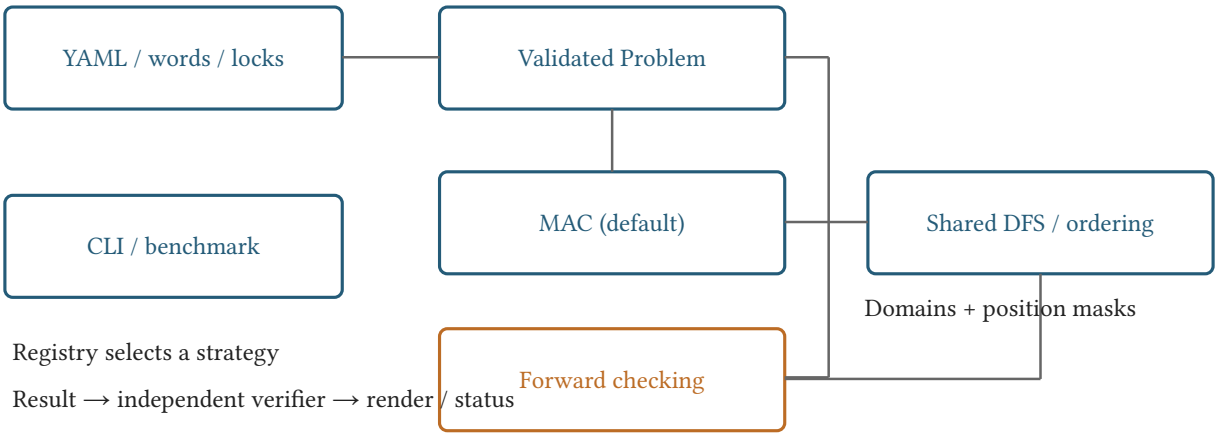


Figure 3: F3. Frontend-independent solver architecture: both strategies share search, ordering, indexes and reversible domain state; each result crosses the independent verification boundary. There is no staging or subproblem decomposition. This is a code-derived schematic; dictionary, seed, budget, machine and zero/censoring conventions are inapplicable.

The code baseline `d9e568f` passed 105 tests, strict mypy over `src` and Black formatting. Tests include exhaustive independent tiny-instance oracles, metadata and lock defects, external thematic words, ñ, seeded equality, cache

restoration, cancellation and deliberately invalid solver outputs. These are correctness checks, not evidence for performance on additional grid corpora.

3 Intellectual Lineage

We distinguish an established **principle** from priority for an exact piece of code. "Established origin" below means the primary work to which we can trace that principle; it does not assert that every cited author was the first to use every elementary data structure. If we could not locate the exact realization, we say so rather than interpreting absence as novelty. Every shipped algorithmic mechanism is classified as established; untraced packaging is conservatively treated as an engineering realization of an established principle, not as a demonstrated new device.

We searched Crossref titles and bibliographic queries for crossword construction, crossword complexity, arc consistency, bitwise table filtering and backtrack programming; we also inspected the arXiv crossword-title feed and the primary metadata/abstracts for the complexity, Dr.Fill and recent clued-solvers papers. The record is summarized in `documentation/research-notes.md`. This was not an exhaustive archival or patent search, and some publisher/DBLP endpoints were unavailable or rate-limited. Claims of **absence from all prior work** would not be justified by this search.

Mechanism in crossbot	Established origin	Deviation / refinement
Slot-word variables and indexed crossing equality	Crossword precedence/search formulations [4], [6]; relation networks [15]	Established. A fixed-pattern feasibility CSP, not pattern creation or clued weighted inference.
Length partition, membership set and stable first-occurrence word IDs	Not found in the surveyed literature for this exact representation. General finite-domain indexing is established [16].	Established indexing principle; stable IDs and duplicate-load accounting are untraced engineering packaging.
Unicode normalization and preserved ñ	Canonical normalization in Unicode Annex 15 [17]	Established. Accent removal excludes ñ; dictionary loading deliberately does not remove accents.
Locked singleton domains, including non-dictionary words	Unary restrictions in relation networks [15]	Established. Problem-local appended IDs implement the dictionary exemption and preserve same-length ID identity.
Integer domains; position/letter masks; intersections	Bitwise arc filtering [18]; reversible bitset table filtering [19]	Established. Plain Python integers, not sparse machine-word sets; masks project words onto letters.
Packed bytearray index compilation	Not found in the surveyed literature for this exact compiler device. Bitset construction is an established principle [18].	Established principle; byte-to-integer materialization is an untraced engineering realization.
FIFO propagation queue with duplicate suppression	Queued arc-consistency filtering [16]	Established. Sources queue slots, not arbitrary binary arcs. No AC-4 support counts [20] or AC-6 residue machinery [21].
Crossing revision from supported letter sets	Arc consistency [16]; bitwise support filtering [18]	Established. Alphabet projections specialize the binary relation; the target uses a union of masks.
Unchanged-letter-signature skip	Event-based propagation and necessary wake-up conditions [22]	Established event-filtering principle. A per-position signature is saved/restored with domains; exact 27-bit packaging was not located.
Union the smaller of supported/absent letters	Not found in the surveyed literature for this exact heuristic; bitwise filtering is established [18].	Established set-complement principle, untraced engineering choice; no new-device claim.
Singleton word deletion from same-length peers	Forward checking [23]; difference constraints discussed by [14]	Established, weaker than matching-based all-different. No Hall-set detection.

Mechanism in crossbot	Established origin	Deviation / refinement
Reversible domain/support trail	State restoration in backtracking [24]; explicit trail machinery [25]	Established. Save whole integer domains and small signature lists, not individual value deletions or Prolog bindings.
MRV slot selection	Fail-first/backtrack ordering [23], [24]	Established. Recomputed popcounts; already singleton slots are omitted.
Unresolved-degree tie-break, then slot index	Connectivity-aware search ordering [23]	Established. Count only nonsingleton crossing neighbours, not global distinctness peers; deterministic final tie.
Least-constraining sampled word score	Value ordering and future-domain freedom [23]	Established LCV principle. Sum logarithms of immediate support counts; independence approximation, no full lookahead. Exact formula's first origin not located.
48-value sample, dense rejection or sparse enumeration	Not found in the surveyed literature for the exact two-regime sampling device. Randomized search is established [26].	Established random-sampling/search principle; thresholds 48, 96 and one eighth are engineering choices, not learned constants.
Seeded pseudorandom stream; sorted sets before shuffle	Mersenne Twister [27]	Established PRNG. Python's `Random` supplies draws; canonical set-to-list conversion avoids hash-order dependence.
Sample first, exhaustive ascending-ID remainder	Enumerative backtracking [24]	Established. Sampling changes priority, never domain completeness within one uninterrupted attempt.
Geometric node-budget restarts	Randomized combinatorial restarts [26]; schedule theory [28]	Established. 400-node initial quota, doubling with exponent capped at 12; not the universal Luby schedule.
Forward-checking alternative	Forward checking [23]	Established. Reuses the engine but only singleton sources constrain crossings.
Monotonic absolute deadline	Python performance-clock interface [29]	Established engineering API. Cooperative cancellation, not hard real-time preemption.
Independent witness validation and registry boundary	Not found in the surveyed algorithmic literature for this exact interface.	Established software-verification and modularity principles; no novel application or algorithmic claim. No solver-side structural splitting exists.

Table 1: Mechanism-by-mechanism lineage of the shipped code baseline `d9e568f`. This is a source/code comparison, not measurement; dictionary, seed, budget and machine are inapplicable.

3.1 Inexact correspondences and nearby ideas

The bitset correspondence is direct at the set-operation level, but crossbot does not implement Compact-Table's sparse reversible bitset representation, active tuple bookkeeping or its generic GAC propagator [19]. A word-slot domain and its letter projections are enough for a shared-character binary relation. This reduces code and domain operations, but should not be described as a new table-consistency algorithm.

Signature caching is an event condition: removing words need not change the set of letters available at a crossing. Schulte and Stuckey analyze efficient propagation-engine wake-up conditions [22]. Our cache is simpler and problem-specific. It differs from remembering a particular support tuple, as in residue-based consistency algorithms [21], and from maintaining a support count for every value [20]. The complement-union choice and bytearray compilation are ordinary set algebra and representation engineering; their exact code idioms have no recovered paper origin. They are not tagged **new device**.

The LCV score treats neighbour-domain counts as independent. That is inexact on cyclic crossing graphs and ignores correlations within a dictionary word. It is much weaker than the probabilistic message passing of Pearl [30] or the clue-weighted beliefs of PROVERB and Berkeley [7], [10]. Taking logarithms only avoids multiplying large counts; it does not turn this ordering into a calibrated probability model. Restricting expensive ranking

to a bounded sample is an engineering compromise. Randomization preserves a deterministic draw stream for a fixed interpreter/input/seed, but a censored run can execute different amounts of work under different machine loads.

Geometric restart quotas are related to heavy-tailed randomized search [26], not proven optimal for this distribution. Luby's universal schedule has a different sequence and guarantees under assumptions that are not established here [28]. Our cap is explicit and unexercised in the reported experiments. No constant was chosen through a train/held-out optimization campaign; this limits generalization claims about 48 or 400.

Finally, global uniqueness is not a local crossing edge. The distinction matters for decomposition: a region filled independently can still reuse a word chosen elsewhere. Freuder's structural conditions for backtrack-free search [31] and Dechter and Pearl's network heuristics [32] describe useful graph structure, but they do not license treating crossbot's regions as independent. Ginsberg's crossword-specific partition/postprocessing work [9] addresses a richer objective; merely copying a grid partition does not inherit its guarantees. This is a divergence between the visually local crossword graph and general CSP dependence, not a discovery of a new structural theorem.

4 Empirical Evaluation

4.1 Setup and evidence protocol

We ran serially in a Linux 6.18.15 aarch64 virtual machine with Apple vendor identification, five exposed vCPUs and approximately 7.9 GiB RAM; the physical host model is not exposed. The interpreter was CPython 3.14.7. The primary dictionary was `dictionaries/words_alpha.txt`, with 370,105 surviving lines and SHA-256 `3ed0c94610d8bcf7c11bbb49c56aa49c7234d32b66824df91f554169e572da48`. `tools/measure.py` records the source hash, the environment, budgets, seeds, elapsed times and witnesses in `benchmarks/results.json`. Every returned assignment was independently verified before recording success. `tools/audit_results.py` subsequently reverified all 225 persisted successful fills; 14 censored/failing runs were retained explicitly across the suite, ablation and alternative-dictionary campaigns.

The complete suite uses seed 0 and a five-second budget for each solver run. Ablations use seeds 0, 1 and 2 at the same budget. Each problem is built once, then shared across strategies; solvers own independent mutable search state. `perf_counter` measures solver wall time, excluding dictionary loading, geometry/index preparation and verification. The primary dictionary load took 0.198 s; once-per-grid preparation summed to 0.883 s across all 88 files. Indexes are lazy and reused, so these preparation numbers are not equivalent to 88 fresh-process compilations. The code source digest is checked during artifact auditing. No timing confidence intervals or multiple-machine measurements were collected; millisecond differences should not be treated as statistically settled.

4.2 Headline results

The default strategy solved **87/88 files (98.86%)**; all 87 successful seed-zero runs were below one second. Thus the under-one-second share of the entire suite is also 98.86% in this solver-time metric. The median over successful runs was 0.036 s, and the worst successful run was Gecode 45 at 0.271 s. Core 87 reached the budget without a fill (measured wall time 5.000134 s, not a zero reading). It is the only default-solver failure. The core and Gecode files sharing numeric IDs are distinct instances; the count is by file, not by unique ID.

Solver	Family	Runs	Solved	< 1 s	Median solved	Worst solved
`mac`	all	88	87	87	0.036 s	0.271 s
`mac`	core	16	15	15	0.035 s	0.222 s
`mac`	gecode	72	72	72	0.038 s	0.271 s
`forward`	all	88	86	86	0.012 s	0.65 s
`forward`	core	16	15	15	0.012 s	0.159 s
`forward`	gecode	72	71	71	0.013 s	0.65 s

Table 2: Complete-suite summary. Medians/worst values exclude failures, but the denominators and subsecond counts include all selected files. No zero timings occur; failures remain censored, not imputed as solved. Data: `words_alpha.txt` (370,105 surviving lines), seed 0 unless stated, 5 s per run; Apple-vendor aarch64 Linux VM (5 vCPUs, 7.9 GiB), CPython 3.14.7.

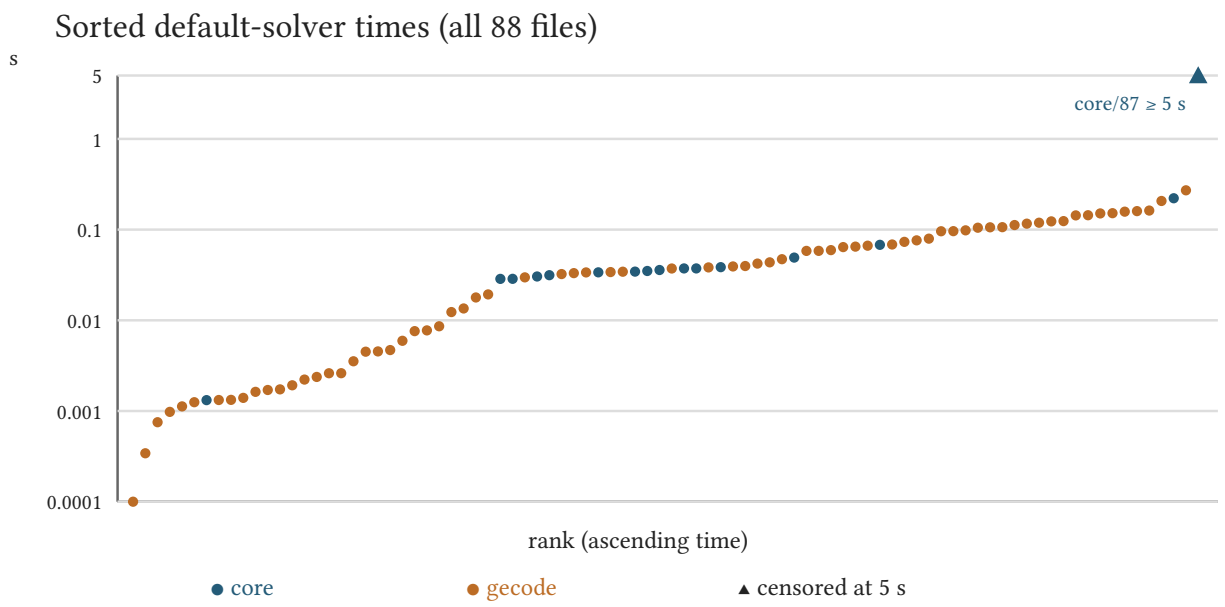


Figure 4: F4. Sorted default-solver wall times for every supplied file, logarithmic vertical scale; colours distinguish grid families. Data: `words_alpha.txt` (370,105 surviving lines), seed 0 unless stated, 5 s per run; Apple-vendor aarch64 Linux VM (5 vCPUs, 7.9 GiB), CPython 3.14.7. The failed core 87 run is a triangle at the 5 s budget. Positive readings below 0.0001 s are plotted at that floor; there are no measured zero readings.

The all-grids exit condition remains unmet. More importantly, these measurements do not establish the requested target for **cold-process end-to-end compiler time**: the solver-time convention comes from the observable CLI contract. The separate preparation and load figures disclose this boundary rather than silently crediting free indexing. No unsolved file was dropped from Figure 4 or the README's complete table.

4.3 Cross-solver comparison on the harder family

On Gecode, `mac` solved 72/72 and `forward` 71/72 under the seed-zero protocol in Table 2. Forward checking's solved-run median was 0.013 s against MAC's 0.038 s, but it timed out on Gecode 45. Its full-family elapsed sum was 8.461 s versus MAC's 4.040 s; a low conditional median therefore does not imply a better regression-gate total. Both strategies failed core 87. Forward checking was also quicker on many core successes; the same domain representation and ordering isolate the principal strategy difference rather than conflating it with a new implementation stack.

The evidence is consistent with a propagation trade-off: forward checking avoids nonsingleton alphabet scans, while MAC invests in earlier detection of incompatible crossing domains. This is not proof that MAC dominates every grid or dictionary. The time limit, one suite seed and different failure sets prohibit an unconditional speedup claim.

4.4 Structural findings

All supplied open cells are covered by some derived slot, so the isolated-open-cell rejection is exercised by tests but never changes a suite outcome. There are no locks in the performance suite; their normalization, membership exemption and mutual-consistency checks are likewise not performance-tested here. The support cache has measurable wall-time impact in the controlled ablation below, but does not change assignments or assignment counts. No feature for matching-based uniqueness, learned nogoods or structural splitting is shipped, so apparent visual regions cannot be credited as an implemented optimization.

Restarts activated in only two complete-suite MAC seed-zero runs: core 90 and unresolved core 87. The great majority of successful runs therefore do not test the restart policy. Its utility is better assessed by the seeded

hard-instance study; attributing the entire suite's subsecond results to restart theory would exaggerate the evidence.

4.5 Attribution study

We selected the three slowest **known-filled** default seed-zero cases from the suite: core 90 and Gecode 45 and 49. Their verified witnesses establish that these instances have fills. Core 87 was excluded from the attribution denominator because its fill status is open, not because its runtime was favourable. We toggled each default feature individually, preserving the other choices, dictionary and five-second budget, and ran seeds 0, 1, 2 for nine runs per configuration. Total time includes measured censored time. `nodes` is attempted assignments and includes restarts; it is a common work statistic, but a MAC node and a forward node have different costs.

Configuration	Total time	Verified	Assignment nodes
`default`	2.004 s	9/9	2466
`no_arc_consistency`	10.956 s	8/9	149644
`no_value_ordering`	6.266 s	9/9	6860
`no_restarts`	6.557 s	8/9	10035
`no_degree`	2.037 s	9/9	2600
`no_support_cache`	2.331 s	9/9	2466

Table 3: Isolated-feature attribution on core 90 / Gecode 45 / Gecode 49. Data: `words_alpha.txt` (370,105 surviving lines), seed 0 unless stated, 5 s per run; Apple-vendor aarch64 Linux VM (5 vCPUs, 7.9 GiB), CPython 3.14.7. Seeds 0, 1, 2. Time includes failures at their measured deadlines; no zero imputation.

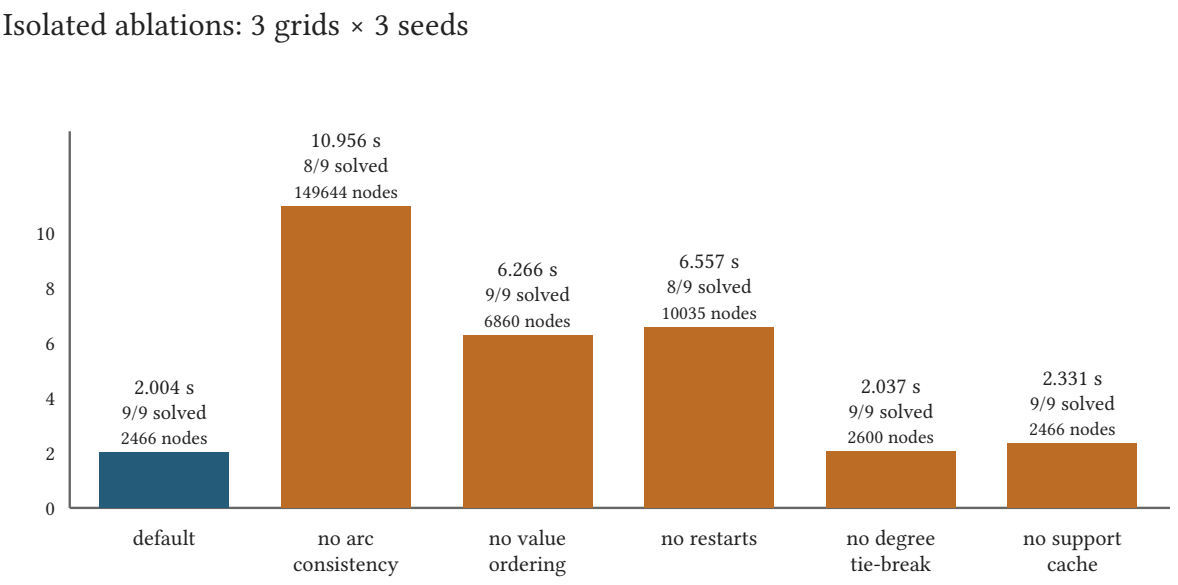


Figure 5: F5. One bar per isolated configuration, annotated with total elapsed solver time, verified runs and assignment nodes. Data: `words_alpha.txt` (370,105 surviving lines), seed 0 unless stated, 5 s per run; Apple-vendor aarch64 Linux VM (5 vCPUs, 7.9 GiB), CPython 3.14.7. Same three grids and seeds 0, 1, 2 as Table 3. Positive totals are plotted linearly. Failed runs contribute their measured censored time; no zero readings occur.

Removing arc consistency increased the measured total from 2.004 to 10.956 s and lost Gecode 45 seed 0. Removing restarts lost Gecode 45 seed 2 and raised the total to 6.557 s. Without value ordering all nine runs still solved, but the total rose to 6.266 s. Restarts and value ordering interact: ranking them by these totals alone would obscure that one prevents a timeout while the other reduces work across several successes. With reliability first and total elapsed time second, the evidence ranks **arc consistency, restarts, then value ordering** as the consequential features on this selected set.

Removing the support-signature cache left exactly 2,466 assignment nodes, identical to the default, but increased total time to 2.332 s. Relative to the uncached measurement, caching saved approximately 14.1% wall time; this

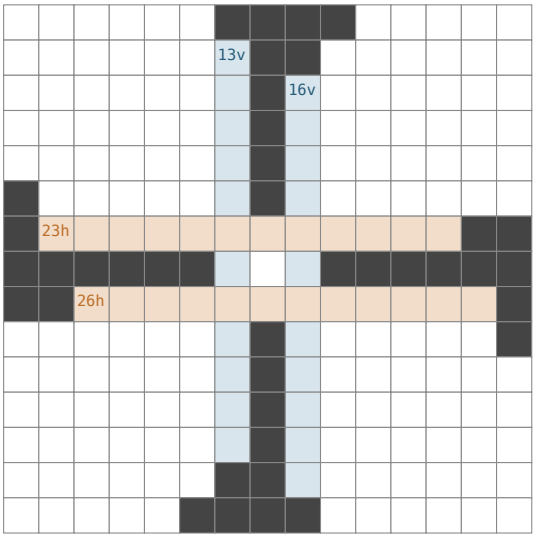
is consistent with avoiding redundant unions rather than pruning extra branches. Degree-tie removal changed 2,466 nodes to 2,600 and total time to 2.037 s. Its 1.6% aggregate timing difference is small enough that further repetitions are needed before ascribing a robust gain.

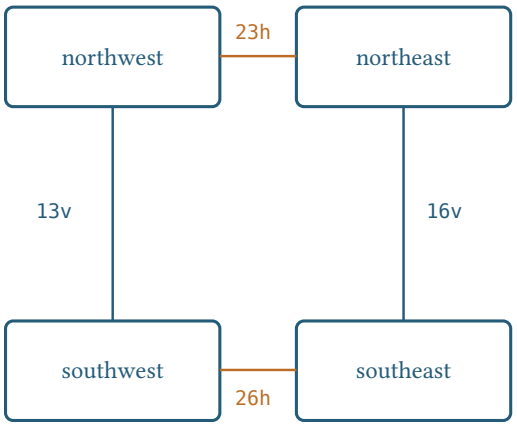
There is no unshipped configuration that beats the default on the **aggregate nine-run reliability-and-time metric** in this campaign. But the counterexample at instance level is explicit: disabling arc consistency solved the three core 90 seeds in a summed 0.050 s, versus the default's 0.377 s. Disabling restarts also marginally reduced that core-only sum, while failing a Gecode run. The default is a corpus-level reliability choice, not a per-instance oracle. These instances were selected from the same suite used for headline reporting; the ablation is an attribution study on hard known fills, not an independently held-out tuning result.

4.6 The unresolved instance and a boundary proxy

Core 87 is 15 by 15, with 185 open crossing cells and 54 slots: 32 of length seven, 12 of length six, four of length five, four of length twelve and two of length three. Four dense regions communicate through the long slots. The English dictionary supplies 41,998 distinct seven-letter words and 29,124 twelve-letter words, but independent word counts do not guarantee compatible dense blocks.

Core 87: four dense regions and four long linking words





Schematic omits two central 3-letter slots.

54 slots; 185 crossings; no verified fill found in the recorded runs.

Figure 6: F6. Exact blocked/open pattern of unresolved core 87, highlighting its long linking slots; the right panel is a region-level schematic, not a decomposition used by the solver. Data: `words_alpha.txt` (370,105 surviving lines), seed 0 unless stated, 5 s per run; Apple-vendor aarch64 Linux VM (5 vCPUs, 7.9 GiB), CPython 3.14.7. The structure is derived from YAML, not timed. Its recorded solver readings are censored at 5 s; no zero-time or solution-status inference is drawn from the schematic.

The final campaign allocated 55 seconds of solver budget to core 87: two suite strategy runs and nine default runs across English, UK English and Spanish dictionaries with seeds 0, 1, 2. All eleven reached their deadlines without a fill. The alternative dictionary load counts were 225,349 and 109,419 surviving lines respectively; their exact fingerprints are in `other_dictionaries` in the raw artifact. This establishes failure of these runs, not absence of a fill. Its status is **open**: neither a witness nor an unsatisfiability proof is known to this implementation.

For a deliberately limited estimate of proximity to a feasibility boundary, we used a first-moment constrainedness surrogate inspired by phase-transition studies [33], [34]. Let each slot be independently uniform over its original length domain; let each crossing's agreement probability be the dot product of its two empirical position-letter frequency distributions. Then

```
E_proxy = product(slot domain sizes) * product(crossing agreement probabilities)
q_proxy = exp(-log(E_proxy) / number_of_slots)
```

On core 87 and the primary English dictionary, this gives `log10(E_proxy) = 18.179` and `q_proxy = 0.461`, regenerated from masks by `diagrams/generate.py`. Under the **surrogate's independence assumptions**, uniform thinning to about 46% of each domain would reduce its expected count to one. This is not a measured threshold: within-word letter correlations, cycles and distinctness violate the approximation. The surrogate predicts abundance despite every censored attempt failing, so it provides no defensible estimate of the **actual** distance to a fill/no-fill boundary. Controlled thinning around a known witness and substantially stronger decision search are needed to locate that boundary. We report this failed diagnostic rather than assigning an unsupported "near-critical" label.

4.7 Negative exploratory results and limits

Before the final controlled campaign, we tried full-propagation candidate lookahead, weighted-degree ordering, high-degree regional ordering, crossing-letter branching and coarse conflict-directed explanations. Related published mechanisms are weighted-degree search [35] and conflict-directed backjumping [36]. These prototypes did not produce a verified core 87 fill in their recorded five-second runs; the direct Spanish baseline also failed a recorded 120-second attempt. We reverted the prototypes to keep the shipped solver small. The archived `benchmarks/exploration` logs preserve these negative outcomes, dictionaries, seeds, quotas and work counts, but do not preserve every intermediate source version. Therefore they are **not** reproducible feature ablations and cannot support a controlled ranking of those techniques. The final isolated study in Table 3 is the evidence for shipped-feature attribution.

The single machine, chosen dictionary, reused indexes, suite-derived hard-set selection and limited seed sample restrict generalization. Neither statistical robustness on unseen dense grids nor superiority to an optimized external CSP engine has been measured. The all-grid goal and a reliable witness for core 87 remain the primary missing evidence.

5 Novelty Assessment

5.1 What is established

Slot-based crossword CSPs and deliberate search ordering are established in construction and filling literature [4], [6]. Arc consistency, forward checking and fail-first search are established general techniques [16], [23]. Bitset support filtering and reversible table representations predate this code [18], [19]. State restoration, event-sensitive propagation, randomized search and restart scheduling are also established [22], [25], [26], [28]. The same classification is used in Table 1: an implementation independently arriving at a known idea does not make that idea novel.

The report's dictionary-exempt locks, Unicode handling, stable IDs, registry and verification boundary are requirements and software engineering devices. Their combination is useful for a conforming CLI; usefulness alone is not an archival novelty claim. The sampled log-support value score is an application of established LCV freedom estimation, not a new inference principle. In particular it does not implement the probabilistic models of clued systems [7], [10].

5.2 Is the combination new for this problem?

The package combines Python integer domains, per-position alphabet masks, signature-sensitive propagation, bounded LCV ranking and geometric restarts under a verified-input/output contract. We found no primary publication documenting precisely this implementation tuple and these constants for the supplied assets. However, crossword-specific search papers already combine heuristic variable/value choice with propagation [6], and modern clued filling combines still richer inference and search [9], [10]. We therefore cannot defend "first use of this combination" from the bounded literature search. The defensible contribution is a inspectable implementation and reproducible negative/positive measurements, not a demonstrated novel combination.

5.3 Candidate individual devices

Signature-sensitive filtering. Its nearest published neighbour is event filtering in propagation engines [22]. The alphabet projection is a natural event abstraction for crossword equality. Verdict: an established principle with a problem-specific engineering realization, not a novel principle or a proven novel application.

Packed bytearray construction and smaller-complement unions. Their nearest neighbours are bitwise support and reversible table algorithms [18], [19]. The exact idioms were not traced to a publication, but they implement ordinary set construction and complement algebra. Verdict: engineering devices not worth a paper on their own; no demonstrated novelty.

Two-regime sampled LCV and deterministic draw accounting. The neighbours are LCV ordering [23] and randomized combinatorial search [26]. The thresholds trade sorting effort against sample coverage. Verdict: engineering parameterization of established principles. No training protocol supports a claim that these constants are optimal, and the negative grid 87 outcome is evidence against a universal sufficient recipe.

Structural factoring, message-passing value guidance, learned nogoods and matching-based uniqueness are **untested improvement ideas for the shipped system**. We do not claim them as contributions; their established neighbours are identified in Section 6. The reverted exploratory devices likewise do not become novel merely because they were attempted unsuccessfully.

5.4 Is it state of the art?

No state-of-the-art claim is supported: crossbot has not undergone a controlled head-to-head comparison on the same machine, dictionaries, grids, uniqueness rule and budget against published filling systems. The Berkeley and modern language-model results address clue-solving objectives and different candidate distributions [10], [12]; their reported accuracy is not a fill-time baseline for these assets. Our controlled baseline is only the shipped forward-checking strategy, and our default still fails one required grid. Comparisons to unimplemented AC-4/AC-6, generic table engines or Dr.Fill would require new measurements, not literature-title analogy.

6 Improvement Opportunities

6.1 Immediate, evidence-backed changes

Keep the support-signature cache and test its invariants. Its controlled benefit is representation work, not pruning: Table 3 records identical 2,466 assignment nodes with and without the cache and about 14.1% lower total time with it on the selected nine runs. The immediate engineering action is to preserve cache/trail correctness during refactoring. This is the only quantified implementation-overhead improvement isolated by unchanged work counts; it should not be extrapolated to every dictionary or dense instance without repetitions.

Investigate strategy selection, but do not ship a grid-ID rule. On the three core 90 seeds the forward-checking/no-arc configuration consumed 0.050 s against MAC's 0.377 s, whereas the Gecode 45 failure favours MAC. An evidence-backed opportunity is to characterize predictors of this trade-off and validate a scheduler on a held-out corpus. The 0.327 s core-90 saving is an observed upper opportunity for those runs, not a promised suite-wide gain. A portfolio must charge both strategies' work to the same deadline; the best-of-two table is not a free algorithm.

Repeat small-effect measurements before tuning ties. Removing the degree tie-break changed total time by only about 1.6% in Table 3. The immediate action is additional interleaved runs, not an assertion that the tie-break matters greatly. An isolated comparison can quantify timing variance while checking that node differences are reproducible. Changing sample/restart constants requires a declared train/held-out split; this campaign contains no evidence for an optimal alternative value.

6.2 Established engineering upgrades

Stronger all-different filtering. Matching-based pruning can detect a set of slots competing for too few words even when no domain is singleton [14]. Crossbot currently misses these Hall-like failures. A selective implementation for small heavily constrained same-length domains is an established upgrade; its overhead and activation frequency on this corpus remain unmeasured. No percentage speedup is promised from the literature alone.

Weighted conflicts and backjumping with precise explanations. Dom/weighted-degree ordering [35] and conflict-directed backjumping [36] can concentrate search and skip irrelevant decisions. The coarse prototype logs do not establish that these principles are ineffective; they establish only that those implementations did not

solve core 87 within their recorded budgets. A revised experiment must preserve sound explanations through crossing reductions, distinctness deletions and restoration, and compare assignment counts as well as wall time on both families.

Structure-aware search with distinctness retained. Graph-based ordering and clustering [31], [32] and crossword-specific partition ideas [9] provide established starting points. Core 87's four long linking slots suggest caching conditional region feasibility or explicit separator search. Distinctness couples otherwise disjoint crossing regions, so cached witnesses need an interface for used words or a sound recombination check. The required experiment is an exact, generic decomposition with every subproblem's work charged to the common deadline, not four independently claimed successes.

More informative candidate guidance. Belief propagation originates in probabilistic inference [30] and is used in modern clued filling [10]. It could account for correlations beyond immediate support products, but implementing and charging its iterations is essential. Bitwise AC-4/AC-6-style support management and Compact-Table-like techniques offer alternative propagation trade-offs [19], [20], [21]. These are established upgrades, not evidence that they will improve this particular Python equality specialization.

6.3 Research directions

Find a witness or a proof for core 87. The explicit open status is the central outstanding question. A new experiment should include all three supplied dictionaries, multiple seeds, independent witness verification and time-to-first-fill distributions. If a longer-budget solver finds a witness, five-second performance can then be studied without confusing scarcity with implementation failure. If a complete search proves unsatisfiability for one dictionary, the goal still requires investigation of the others. Neither a timeout nor the first-moment proxy is a certificate.

Measure a real feasibility transition. The crude surrogate in Section 4 ignores correlations and predicts many assignments even on the unresolved case. A controlled thinning experiment should start from known-fill dense grids, preserve or explicitly remove witness words, vary dictionary fractions and use a decision method capable of resolving enough unsatisfiable cases. Phase-transition and constrainedness methodology [33], [34] motivates the protocol, but the experiment needs a substantially broader dense-grid corpus before "near boundary" can become a reliable structural label.

Test generalization and objective changes. The supplied dictionaries differ in letter morphology, while clued systems operate on weighted candidates [7], [9]. A new corpus should separate variable-length sparse grids from dense square-like blocks and should include locks, decomposed crossing graphs with word-reuse conflicts and Spanish ñ. Cold-process indexing time, memory peaks and held-out strategy selection must be measured. This would establish which representation and ranking choices transfer; the present seed-zero English suite cannot answer that question.

7 Conclusions

Crossbot implements the specified validation, normalized thematic locks, dictionary loading, solver selection, verified Unicode rendering and benchmark failure gate. Its algorithmic core is small: shared word-ID masks, reversible domains, equality-specific arc consistency, minimum remaining values, sampled least-constraining words and geometric restart quotas. Forward checking changes one propagation choice while reusing the same machinery. Independent output checking prevents internal-domain assumptions from certifying their own mistakes.

The lineage verdict is conservative. Slot CSPs, propagation, value/variable ordering, bitwise support filtering, trailing, event filtering and randomized restarts are established. The exact bytearray compilation, complement-union choice, signature packaging and sample thresholds have no fully located origin in our bounded search, but this is not evidence of a new principle. We claim an inspectable engineering combination and its evidence trail, not archival novelty or a first-of-kind solver.

The empirical hierarchy depends on the metric. Under the declared five-second, seed-zero, English protocol, MAC filled 87 of 88 files; every verified success was below one second and the worst verified time was 0.271 s. The default misses the required all-grid exit condition on core 87. Forward checking has a lower successful-run

median but an additional Gecode timeout; its conditional speed does not outweigh that regression-gate failure by definition. Neither strategy has demonstrated universal superiority.

On the selected three known-filled hard grids and seeds 0, 1, 2, the isolated study is consistent with arc consistency being the strongest reliability/time contributor, followed by restarts and LCV ordering. Signature caching reduced implementation work without changing assignment nodes; degree ties had a much smaller, insufficiently repeated timing effect. Restarts do not activate on most suite successes, and the uncached configuration's identical work count prevents attributing its timing difference to stronger logical pruning. The study also exposes a genuine counterexample: forward checking is better on the measured core 90 seeds. A default can be a reliability choice without being optimal on each instance.

Core 87 is the unresolved bottleneck, not an omitted outlier. Its dense regions and long linking slots suggest a different search structure, but global word uniqueness prevents naive independence. All recorded final attempts across the supplied dictionaries were censored. No witness or unsatisfiability proof is available. A first-moment independence proxy predicts abundant assignments and therefore fails as a calibrated boundary diagnostic. The report makes the negative result visible rather than assigning a convenient difficulty explanation unsupported by a certificate.

The measured position relative to the state of the art is consequently limited: there is no controlled external-system comparison, and modern clue solvers solve a different weighted retrieval-and-fill task. Solver times also exclude input/index preparation; the separately reported load/preparation costs do not substitute for a cold-process full-suite experiment. The repository provides raw seeds, budgets, fingerprints, work counters and successful witnesses so a subsequent comparison can adopt a common protocol.

Concrete next steps are to retain the tested representation/cache invariants, repeat the small-effect measurements, implement a sound generic stronger-search or conditional-region mechanism, and obtain a witness or proof for the outstanding dictionary/grid cases. Only after that should we optimize or learn a strategy selector on declared training and held-out grids and measure cold end-to-end compiler time. Until core 87 can be filled within the common budget, the correct completion statement is partial implementation success with a clearly unmet performance goal.

References

[1] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979. [Online]. Available: https://archive.org/details/computersintract0000gare

[2] L. Gourvès, A. Harutyunyan, M. Lampis, and N. Melissinos, "Filling Crosswords is Very Hard." [Online]. Available: https://arxiv.org/abs/2109.11203

[3] L. J. Mazlack, "The use of applied probability in the computer construction of crossword puzzles," in *IEEE Conference on Decision and Control*, 1973. doi: 10.1109/CDC.1973.269214.

[4] L. J. Mazlack, "Computer construction of crossword puzzles using precedence relationships," *Artificial Intelligence*, vol. 7, no. 1, pp. 1–19, 1976, doi: 10.1016/0004-3702(76)90019-9.

[5] P. D. Smith, "XENO: Computer-Assisted Compilation of Crossword Puzzles," *The Computer Journal*, vol. 26, no. 4, pp. 296–301, 1983, doi: 10.1093/comjnl/26.4.296.

[6] M. L. Ginsberg, M. Frank, M. P. Halpin, and M. C. Torrance, "Search Lessons Learned from Crossword Puzzles," in *AAAI*, 1990. [Online]. Available: https://www.aaai.org/Papers/AAAI/1990/AAAI90-032.pdf

[7] M. L. Littman, G. A. Keim, and N. Shazeer, "A Probabilistic Approach to Solving Crossword Puzzles," in *AAAI*, 1999. [Online]. Available: https://www.aaai.org/Papers/AAAI/1999/AAAI99-032.pdf

[8] M. Ernandes, G. Angelini, and M. Gori, "WebCrow: A Web-Based System for Crossword Solving," in *AAAI*, 2005. [Online]. Available: https://www.aaai.org/Papers/AAAI/2005/AAAI05-224.pdf

[9] M. L. Ginsberg, "Dr.Fill: Crosswords and an Implemented Solver for Singly Weighted CSPs," *Journal of Artificial Intelligence Research*, vol. 42, pp. 851–886, 2011, doi: 10.1613/jair.3437.

[10] E. Wallace *et al.*, "Automated Crossword Solving," in *ACL*, 2022. doi: 10.18653/v1/2022.acl-long.219.

[11] G. Angelini *et al.*, "The WebCrow French Crossword Solver." [Online]. Available: https://arxiv.org/abs/2311.15626

[12] S. Saha, S. Chakraborty, S. Saha, and U. Garain, "Language Models are Crossword Solvers," in *NAACL*, 2025. doi: 10.18653/v1/2025.naacl-long.104.

[13] C. Biemer and S. Cooper, "Dynamic Crossword Difficulty via Reactive Puzzle Construction," in *Proceedings of the 21st International Conference on the Foundations of Digital Games*, 2026. doi: 10.1145/3815598.3815714.

[14] J.-C. Régin, "A Filtering Algorithm for Constraints of Difference in CSPs," in *AAAI*, 1994. [Online]. Available: https://www.aaai.org/Papers/AAAI/1994/AAAI94-055.pdf

[15] U. Montanari, "Networks of constraints: Fundamental properties and applications to picture processing," *Information Sciences*, vol. 7, pp. 95–132, 1974, doi: 10.1016/0020-0255(74)90008-5.

[16] A. K. Mackworth, "Consistency in Networks of Relations," *Artificial Intelligence*, vol. 8, no. 1, pp. 99–118, 1977, doi: 10.1016/0004-3702(77)90007-8.

[17] The Unicode Consortium, "Unicode Standard Annex 15: Unicode Normalization Forms." [Online]. Available: https://www.unicode.org/reports/tr15/

[18] C. Lecoutre and J. Vion, "Enforcing Arc Consistency using Bitwise Operations," in *AAAI*, 2008. [Online]. Available: https://dblp.org/rec/conf/aaai/LecoutreV08.html

[19] J. Demeulenaere *et al.*, "Compact-Table: Efficiently Filtering Table Constraints with Reversible Sparse Bit-Sets," in *Constraint Programming*, 2016. doi: 10.1007/978-3-319-44953-1_14.

[20] R. Mohr and T. C. Henderson, "Arc and Path Consistency Revisited," *Artificial Intelligence*, vol. 28, no. 2, pp. 225–233, 1986, doi: 10.1016/0004-3702(86)90088-5.

[21] C. Bessière, "Arc-consistency and arc-consistency again," *Artificial Intelligence*, vol. 65, no. 1, pp. 179–190, 1994, doi: 10.1016/0004-3702(94)90018-3.

[22] C. Schulte and P. J. Stuckey, "Efficient Constraint Propagation Engines," *ACM Transactions on Programming Languages and Systems*, vol. 30, no. 1, 2008, doi: 10.1145/1290520.1290522.

[23] R. M. Haralick and G. L. Elliott, "Increasing tree search efficiency for constraint satisfaction problems," *Artificial Intelligence*, vol. 14, no. 3, pp. 263–313, 1980, doi: 10.1016/0004-3702(80)90051-X.

[24] S. W. Golomb and L. D. Baumert, "Backtrack Programming," *Journal of the ACM*, vol. 12, no. 4, pp. 516–524, 1965, doi: 10.1145/321296.321300.

[25] D. H. D. Warren, "An Abstract Prolog Instruction Set," technical report Technical Note 309, 1983. [Online]. Available: https://www.sri.com/publication/an-abstract-prolog-instruction-set/

[26] C. P. Gomes, B. Selman, and H. Kautz, "Boosting Combinatorial Search Through Randomization," in *AAAI*, 1998. [Online]. Available: https://www.aaai.org/Papers/AAAI/1998/AAAI98-061.pdf

[27] M. Matsumoto and T. Nishimura, "Mersenne Twister: A 623-Dimensionally Equidistributed Uniform Pseudo-Random Number Generator," *ACM Transactions on Modeling and Computer Simulation*, vol. 8, no. 1, pp. 3–30, 1998, doi: 10.1145/272991.272995.

[28] M. Luby, A. Sinclair, and D. Zuckerman, "Optimal Speedup of Las Vegas Algorithms," *Information Processing Letters*, vol. 47, no. 4, pp. 173–180, 1993, doi: 10.1016/0020-0190(93)90029-9.

[29] V. Stinner, "PEP 418: Add monotonic time, performance counter, and process time functions." [Online]. Available: https://peps.python.org/pep-0418/

[30] J. Pearl, "Reverend Bayes on Inference Engines: A Distributed Hierarchical Approach," in *AAAI*, 1982. [Online]. Available: https://www.aaai.org/Papers/AAAI/1982/AAAI82-032.pdf

[31] E. C. Freuder, "A Sufficient Condition for Backtrack-Free Search," *Journal of the ACM*, vol. 29, no. 1, pp. 24–32, 1982, doi: 10.1145/322290.322292.

[32] R. Dechter and J. Pearl, “Network-Based Heuristics for Constraint-Satisfaction Problems,” *Artificial Intelligence*, vol. 34, no. 1, pp. 1–38, 1987, doi: 10.1016/0004-3702(87)90002-6.

[33] P. Cheeseman, B. Kanefsky, and W. M. Taylor, “Where the Really Hard Problems Are,” in *IJCAI*, 1991. [Online]. Available: https://www.ijcai.org/Proceedings/91-1/Papers/052.pdf

[34] I. P. Gent, E. MacIntyre, P. Prosser, and T. Walsh, “The Constrainedness of Search,” in *AAAI*, 1996. [Online]. Available: https://www.aaai.org/Papers/AAAI/1996/AAAI96-037.pdf

[35] F. Boussemart, F. Hemery, C. Lecoutre, and L. Sais, “Boosting Systematic Search by Weighting Constraints,” in *ECAI*, 2004, pp. 146–150. [Online]. Available: https://dblp.org/rec/conf/ecai/BoussemartHLS04.html

[36] P. Prosser, “Hybrid Algorithms for the Constraint Satisfaction Problem,” *Computational Intelligence*, vol. 9, no. 3, pp. 268–299, 1993, doi: 10.1111/j.1467-8640.1993.tb00310.x.

[37] S. Minton, M. D. Johnston, A. B. Philips, and P. Laird, “Minimizing conflicts: a heuristic repair method for constraint satisfaction and scheduling problems,” *Artificial Intelligence*, vol. 58, pp. 161–205, 1992, doi: 10.1016/0004-3702(92)90007-K.

Appendix A: subsequent unshipped min-conflicts campaign

A.1 Prediction inherited from the frozen main text

The main text identifies core 87 as unresolved and predicts that a materially different search structure or stronger guidance may be needed. It does **not** predict that a larger hidden timeout, a low mismatch count or a local optimum certifies a fill. After freezing sections 1–7, we tested a separate heuristic-repair approach; those sections and their original measurements remain unchanged. This appendix records a negative experiment, not an improvement to the shipped default.

A.2 Method and parameter protocol

`tools/local_search_experiment.py` implements an unregistered min-conflicts-style repair experiment, related to Minton and colleagues' established heuristic [37]. It begins with independent dictionary words of each required length while maintaining global uniqueness. At each step it chooses uniformly among slots with at least one disagreement. A replacement word must not already occur in another slot. A bitset dynamic program computes domains grouped by exact Hamming disagreement with the current crossing letters: matching a letter leaves its cost unchanged; mismatching advances to the next cost bucket. The first nonempty bucket gives all minimum-conflict replacements; a uniform random live ID breaks ties. With noise probability `p`, the algorithm instead picks an arbitrary available word. After 2,000 repairs it restarts from a new distinct random assignment.

We selected `p = 0.02` and `p = 0.10` as two exploratory, fixed low/high noise settings before this campaign. Neither was trained or promoted based on the outcomes. The sole test case was core 87; there is no train/held-out split and no generalization claim. Each setting used seeds 0, 1, 2, the three supplied dictionaries and five seconds per run on the same Apple-vendor aarch64 VM and CPython 3.14.7 as the main campaign. All time, including restarts and repair selection, is charged inside that solver budget; loading/indexing is excluded as before. The script checks any zero-conflict assignment independently with `verify`. No zero-conflict assignment was returned.

This experiment does not support locks, completeness or unsatisfiability proofs and is not registered as a shipped solver. Reproduction command: `uv run python tools/local_search_experiment.py`. It writes `benchmarks/local-search.json`. The objective counts each mismatched crossing once; its "steps" count replacements, not the default engine's assignment nodes. These work units must not be compared as if they had identical cost.

A.3 Measurements and comparison

Dictionary	Noise	Time	Repair steps	Best errors
`words_alpha.txt`	0.02	15 s	176152	17
`words_alpha.txt`	0.1	15 s	176956	27
`uk220k.txt`	0.02	15 s	217261	19
`uk220k.txt`	0.1	15 s	234005	33
`words.es.rae.txt`	0.02	15 s	306829	17
`words.es.rae.txt`	0.1	15 s	310259	28

Table 4: Subsequent core-87 min-conflicts experiment: each row sums three seeds 0, 1, 2 at 5 s each, same machine/interpreter as the main report; dictionary counts 370,105 / 225,349 / 109,419 respectively. Every run is censored; no zero-time imputation. "Best errors" is the minimum recorded crossing disagreements, not a success or boundary certificate.

All 18 runs timed out without a verified fill. The smallest recorded disagreement count was 17 crossing errors; none reached zero. The main default comparison also timed out for core 87 across all three dictionaries and seeds 0, 1, 2. This is no solved-run improvement over that common dictionary/seed/budget baseline. Lower repair noise reached smaller mismatch minima in this campaign, but that cannot be translated into a measured success probability or a feasibility-distance estimate.

A.4 Verdict, regression status and next action

The experiment did not reach the missing exit condition and was not promoted into the production registry. The production source fingerprint and both shipped strategies are unchanged, so no algorithmic regression to the rest of the suite is implied; we did not re-time unrelated grids for an unshipped script. The existing 105 tests, strict mypy check and Black check remained passing after the campaign, and persisted baseline fills remained independently verifiable. The report and bibliography compile without warnings after adding this appendix.

The next action is still to obtain a witness or sound stronger-search result for core 87. A local-repair attempt with many inexpensive moves has now been measured, but a low mismatch count is not a replacement for a solution. No new principle or state-of-the-art claim follows from this negative campaign.